Crustify: A Multi-Agent Automated Pipeline for Porting C to Rust

Marius Momeu Koushik Sen

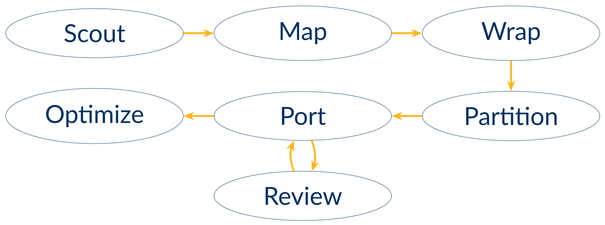
UC Berkeley

Motivation

C remains one of the most widely deployed systems languages, yet memory safety vulnerabilities account for a large fraction of critical CVEs. **Rust** offers strong guarantees — no buffer overflows, no use-after-free, no data races — at comparable or superior performance.

- Manual porting: large teams need years of labor for a single codebase
- Traditional methods (`c2rust`) produce *unsafe* Rust that bypasses its lifetime and ownership invariants
- LLM agents understand both languages and employ powerful software engineering capabilities – can they close the gap?

Crustify's Port Workflow



crustify-crate: C-Compatible Smart Pointers

A key challenge in safe C-to-Rust porting is accessing C types (`structs, arrays`) in Rust without using *raw pointers* or modifying their layout. `crustify-crate` provides *idiomatic smart pointers* that can wrap C refs via layout-compatible, zero-cost, abstractions—this allows Rust's compiler to enforce safety invariants on them without introducing *undefined behavior*:

- `CType<T>` – wraps a C struct T via `#[repr(transparent)]` and `UnsafeCell<MaybeUninit<T>>`
- `CBox<T>` – exclusive ownership, drops via T's `dtor`
- `CArc<T>` – shared ownership, drops via T's `down_ref`, clones via T's `up_ref`
- `CVec<T, S>` – array with scalar/non-scalar elements, drops via the array's `dtor`, cleans up elements via strategy S that calls T's `dtor`
- `CStr` – `NUL`-terminated strings, drops via the string's `dtor`

LLM agents or human devs can use these primitives to enable Rust's `RAII` on C `structs/arrays/strings` without introducing UB hazards or breaking interop with C code.

Crustify V1 — Single-Agent Pipeline

Workflow

- Agent backend: `KISS AI` — Python orchestration, auto-continuation, access to basic tools (Bash/Read/Write/Edit)
- Model: Opus 4.5, 200K context, high thinking effort
- Pipeline: single-agent, simple workflow: `Scout` → `Port` → `Test` → `Fix`

Prompt

- *"You are a C-to-Rust expert .. Scout codebase .. Setup Rust crate .. Port C files to Rust .. Build, test, fix .. Write FFI exports .. You MUST USE idiomatic safe Rust; `unsafe` is only allowed for FFI .."*
- No project-specific hints were given

Experiment 1 — Small/Medium Codebases

- C modules from 12 real-world projects (**28K LoC**): `cJSON` (3.8K LoC), `stb_perlin` (500 LoC), `SPHINCS+` (5K LoC), `curl` (50 LoC), etc.
- All modules had I/O assertion tests that exercised the target

Results

- Completion: finished all tasks, passing all tests
- Cost: $174 in 8h; avg: ≈$4/task, ≈10 min/task
- Safety: no `unsafe` in core logic, only for FFI (`#[unsafe(no_mangle)]`)

Experiment 2 — Large Codebases

- OpenSSL's `libssl` (>100K LoC) — TLS handshake state machine, high CVE exposure (the *Heartbleed* bug); complex type relationships and lifecycles, tight dependency on `libcrypto` (>350K LoC)

Results

- Completion — gives up mid-task, calls C via FFI, forgets to build/test
- Cost — expensive and slow due to overthinking, lack of plan
- Safety — defaults to `unsafe` and raw pointers (`*mut T`)

Crustify V2 — Multi-Agent Pipeline (Beta)

CrustifyScout (Sonnet 4.6)
Input: Codebase (directory, URL)
Task: Identify build system, test suite, API surface, composition, pointer/lifetime patterns, attack surface; await human approval for port scope
Output: Structured manifest

→

CrustifyMap (Opus 4.7)
Input: Manifest
Task: Identify in-scope files, types, functions, globals; map to `ctors/dtors/ops`; use `CodeQL` + `ripgrep`
Output: Structured dependency graph

↓

CrustifyWrap (Opus 4.7/Sonnet 4.6)
Input: Dependency graph
Task: Generate safe Rust wrappers via `crustify-crate`; bindgen FFI bindings; implement `Drop`/`Clone`; add `\\SAFETY` comments; follow `DISCIPLINE.md`
Output: Rust wrapper crates

←

CrustifyPartition (Sonnet 4.6)
Input: Dependency graph + wrapper crates
Task: Topo-sort symbols of graph; partition into ~1K LoC waves; identify parallelization opportunities
Output: Ordered port waves

↓

CrustifyPort (Opus 4.7/Sonnet 4.6)
Input: Port wave + wrapper crates
Task: Translate C to safe Rust per DISCIPLINE.md; ban unsafe/raw pointers; build C+Rust; run C tests; fix; export FFI shim
Output: Safe Rust port

⇄ *fix*

CrustifyReview (Opus 4.7)
Input: Safe Rust port
Task: `LLM-as-a-Judge`: verify builds + tests pass, `DISCIPLINE.md` followed, no `unsafe`/raw pointers outside FFI shim, no UB
Output: Approval or spawn `CrustifyPort` to fix gaps

Workflow

– `CLI` tool + `Python` orchestration
– Each `Crustify` agent uses `KISS AI` as agent backend for basic tooling and auto-continuation
– Stages can run independently or batched (i.e. whole pipeline)
– `HIL` can review output, ask follow-ups, decide when to proceed to next stage

Evaluation – OpenSSL's `libssl`

– LoC: ≈8.5K of C (`statem` subsystem) → ≈11K of Rust
– Functionality: all enabled 4K C tests pass
– Cost: ≈$1.5K in ≈10 days
– Wrapped 46 C structs (`libcrypto/libssl`), 16 can be `CArc`, 21 can be `CBox`, 9 borrow-only
– Safety: only 15% LoC in `unsafe` blocks or raw pointers remaining!

Want to Crustify your codebase? Reach out!

marius.momeu@berkeley.edu